ACBS v2: Important Vibecoding Prompts (Project Edition)

These are the 20 standard prompts filled in for the Auto Classifier Benchmarking System v2 (package acbs): the 8 classic models, the 9 AutoML frameworks, the two-tier benchmark, and the code that already exists. Run them in order. Prompts 1 to 6 create the six documents (save each in the repository root and approve it before the next). Because the repository already exists, prompts 1 to 8 document and align the current code instead of starting from nothing. Prompt 9 is reused once per feature (the feature queue is under it). Prompt 15 is used only when something breaks.

1. PRD Creation

You are a senior Product Manager and Software Architect experienced in taking products from idea to production.

CONTEXT:
I want to build: Auto Classifier Benchmarking System v2 (Python package acbs), a web application and command-line tool that benchmarks classification options on a user's tabular dataset and recommends the most suitable one for the user's priorities.

What it does: it trains and tunes 8 classic classifiers (Logistic Regression, SVM, Decision Tree, Random Forest, XGBoost, Gradient Boosting, Naive Bayes, KNN) plus a majority-class baseline (Tier 1), and runs 9 AutoML frameworks (Tier 2): AutoGluon, Auto-sklearn (including Auto-sklearn 2), FLAML, GAMA, H2O AutoML, LightAutoML, MLJAR, NaiveAutoML and TPOT. Every contender runs under one fair protocol (same split, seed, CPU and memory limits, time budget, and central scoring from predictions). Contenders are compared on accuracy, precision, recall, F1, ROC-AUC, training time, prediction time, budget used and memory. The user's weights and constraints produce a ranking, a recommended option with a plain-language explanation, and an answer to the question: is AutoML worth it for this dataset? Results appear as interactive Plotly charts and downloadable reports.

Current state: v1 (classic tier only) already exists: leak-free preprocessing pipelines, stratified 5-fold cross-validation, Optuna and randomized tuning, per-model failure isolation, min-max and TOPSIS ranking, Pareto and sensitivity analysis, template-based explanations (no LLM), SQLAlchemy storage (SQLite), a FastAPI backend, a Streamlit prototype (to be replaced), a Typer CLI and 28 passing tests. v2 adds the AutoML tier and replaces Streamlit with an HTML, CSS and JavaScript frontend.

Research basis: the AMLB benchmark (Gijsbers et al., JMLR 2024) and our literature review. Gaps this project addresses: no integrated recommendation layer, narrow single-metric evaluation, inconsistent efficiency reporting, fragmented domain coverage (healthcare, finance, fraud, customer analytics), and static reporting.

Setting: a two-person university project (Alliance School of Advanced Computing) that must deliver a working system, a research paper and a demo. Deadline: [I WILL CONFIRM].

TASK:
Create a complete PRD.md that will become the source of truth for the coding agent. Describe what already exists as existing requirements and mark everything new in v2 clearly.

Include:
- Problem, value proposition and target users (ML practitioners and students choosing between classic models and AutoML tools; researchers and reviewers who need reproducible comparisons)
- Core user journeys and user stories (upload data, choose contenders, budget and priorities, monitor a long run, read the recommendation, change priorities without retraining, compare runs, export a report or model)
- Features and functional requirements (numbered, each marked EXISTING or NEW)
- Non-functional requirements (reproducibility, fairness of comparison, robustness when a framework fails, time and cost transparency, platform limits, safe handling of uploaded data)
- Pages/screens (Home, Datasets, New Benchmark, Run Monitor, Results, Model Detail, History, Frameworks, Methodology)
- Roles and permissions (state clearly whether accounts are needed; my assumption is a single local user)
- Data requirements (binary and multiclass tabular datasets in CSV, Excel or Parquet; experiment records; predictions; model artifacts)
- Integrations (OpenML, UCI and Kaggle as dataset sources; the 9 AutoML frameworks run in isolated environments; Docker; Java for H2O)
- Loading, empty and error states (including framework statuses ok, failed, timeout, skipped, unavailable and unsupported)
- MVP scope, out-of-scope items (for example multilabel, multimodal, regression, deep learning, accounts, cloud scaling, live prediction serving) and measurable success criteria

RULES:
Do not write code. Do not invent unnecessary features. Avoid over-engineering. Clearly label assumptions and flag decisions requiring my approval (single-user versus accounts, plain HTML/CSS/JS versus a framework, supported operating systems, default time budgets).

OUTPUT:
Return only clean Markdown ready to save as PRD.md.

2. DESIGN.md

You are a senior Product Designer and UX/UI Design Systems expert specializing in production web applications.

CONTEXT:
Use PRD.md as the product source of truth. The interface is a multi-page static website written in plain HTML, CSS and JavaScript (ES modules, no build step, no framework), one HTML file each for Home, Datasets, New Benchmark, Run Monitor, Results, Model Detail, History, Frameworks and Methodology. It is served by FastAPI at the same origin and talks only to the JSON API under /api. The design must be achievable with CSS custom-property tokens and a small shared component library written in vanilla JS. The product is a research workbench, not a marketing site.

TASK:
Create DESIGN.md as the single source of truth for the application's visual design and UX.

Define:
- Visual direction and design principles (the answer comes first, never hide cost, be honest about failures, explain every chart)

- Colors and typography (one accent, neutral greys, status colors used only for status, tabular numbers for metrics)
- Spacing and layout system (page width, grid, section spacing, sticky summary panels)
- Reusable components (tier badge Classic versus AutoML, status chip for 8 statuses, metric tile, recommendation card, leaderboard table, chart frame with caption and data fallback, cost banner, priority panel)
- Navigation and forms (sidebar order, active-run indicator, the New Benchmark form with live cost estimate, dataset upload with validation messages)
- Loading, empty and error states (long-running jobs, unavailable frameworks, failed runs, API down)
- Interactions and animations (live re-ranking without retraining, periodic progress refresh, no decorative motion)
- Responsive behavior (desktop first, usable at 1024 px, phone layout explicitly not a goal)
- Accessibility requirements (contrast, color never the only signal, keyboard use, text alternatives for charts)
- Page-level layout guidance for each of the 9 pages, including the chart set on Results and the shared Plotly template

RULES:
Prioritize usability and consistency. Avoid generic AI-generated SaaS design and unnecessary decoration. Define reusable rules, not one-off styling. Keep every rule implementable in plain CSS and vanilla JS without a build step; if a rule needs a workaround, say so.

OUTPUT:
Return only clean Markdown ready to save as DESIGN.md. Do not write code.

3. ARCHITECTURE.md

You are a senior Software Architect specializing in secure, scalable and maintainable applications.

CONTEXT:
Use PRD.md and DESIGN.md as the source of truth. The existing codebase (src/acbs, app, tests) must be treated as the starting point; inspect it and describe the architecture as it should be, marking what exists and what is new. Known facts: Python 3.11+ (tested on 3.12), scikit-learn, XGBoost, imbalanced-learn, Optuna, Plotly, Jinja2, FastAPI, a vanilla HTML/CSS/JS frontend (replacing the Streamlit prototype), SQLAlchemy 2.0 with SQLite for development and PostgreSQL for deployment, Alembic, Typer. The 9 AutoML frameworks have conflicting dependencies, so they cannot share one Python environment with the app.

TASK:
Create ARCHITECTURE.md defining exactly how this product should be built.

Include:
- Recommended tech stack with reasoning (keep what exists unless there is a strong reason)
- Frontend/backend architecture (static HTML/CSS/JS frontend served by FastAPI, JSON API under /api, service layers, no ML logic in the frontend or routes)
- Two-tier benchmark design: Tier 1 classic models inside the app environment, Tier 2 AutoML frameworks each in its own isolated environment, called through a subprocess worker with a typed task/outcome contract

- Runner backends: local virtual environments first, Docker second, remote machines as future work (AMLB uses local, container and AWS runners)
- Data flow from upload to recommendation (profile, split once, run tiers sequentially, score centrally from predictions, rank, explain, store, report)
- Folder structure (existing src/acbs modules plus automl/, worker package, envs/, configs/)
- Database interaction and background job execution (progress persisted, restart recovery, cancellation)
- Authentication/authorization (state the MVP model; my assumption is none beyond an optional shared token)
- API architecture (REST, long jobs as start-then-poll)
- External services (dataset sources, Java for H2O, Linux requirement for Auto-sklearn)
- AI architecture: state explicitly that explanations are deterministic templates and no LLM service is used
- Environment configuration (data directory, environments directory, database URL, limits)
- Error handling (status taxonomy, timeouts with grace period, process-group kill, orphan cleanup)
- Security boundaries (uploads, subprocess execution, serialized model artifacts, local ports opened by frameworks)
- Testing and deployment approach (FakeAdapter tests, adapter conformance suite, docker-compose)

RULES:
Choose the simplest reliable architecture. Avoid unnecessary dependencies, microservices or infrastructure (no message queues or orchestration unless you justify them against the project scale). Separate MVP requirements from future scalability. Frameworks must never be installed into the main application environment.

OUTPUT:
Return only clean Markdown ready to save as ARCHITECTURE.md. Do not write code.

4. DATABASE.md

You are a senior Database Architect and Backend Engineer experienced in designing production-grade application databases.

CONTEXT:
Use PRD.md and ARCHITECTURE.md as the source of truth. A v1 schema already exists (datasets, experiments, model_runs, recommendations) created with Base.metadata.create_all; the Alembic versions folder is empty. Inspect src/acbs/storage first and document the current schema, then define the v2 schema as a delta.

TASK:
Create DATABASE.md defining the complete data model.

Include:
- Database technology (SQLite for development, PostgreSQL for deployment, SQLAlchemy 2.0, Alembic)
- Tables and columns with types, including v2 additions: on model runs the tier (classic or automl), framework, framework version, budget, budget used, cores, peak memory, model summary, framework leaderboard and a log tail; on experiments the hardware description, outer

fold and progress fields
- Where settings live (table or configuration file) with a recommendation
- Primary/foreign keys
- Relationships and ER diagram (Mermaid)
- Constraints and validation (allowed status values, unique dataset hash, cascade rules)
- Indexes based on real queries (history list by date, dataset and status; runs by experiment; dataset lookup by hash)
- Ownership and authorization (state that MVP has no per-user ownership if that is the decision)
- Row-level security where applicable (state if not applicable)
- Sensitive-data handling (uploaded datasets may contain health or financial data: local storage only, deletion, never log data rows)
- Files stored outside the database (datasets as Parquet, predictions, model artifacts, reports) with path and hash references
- Migration and seed strategy (initial migration for the existing schema, then a v2 migration; seed the built-in dataset registry)
- Backup/recovery considerations (database file or dump plus the results directory, and a restore check)

RULES:
Design only required data. Avoid duplication, unnecessary tables and premature optimization. Ensure the schema supports the defined user journeys, including restart recovery of interrupted experiments and re-ranking without retraining.

OUTPUT:
Return only clean Markdown ready to save as DATABASE.md. Do not write application code.

5. API.md

You are a senior Backend Architect and API Integration Engineer experienced in production systems.

CONTEXT:
Use PRD.md, ARCHITECTURE.md and DATABASE.md as the source of truth. A FastAPI backend already exists with endpoints for dataset profiling, experiment launching, live status polling, instant re-ranking and report export. Inspect src/acbs/api first and document existing endpoints as they are, then specify the new ones.

TASK:
Create API.md defining how the application communicates internally and with external services.

Include:
- Endpoints, methods and purpose: existing (datasets, experiments, status, results, recommendation, rerank, report, history, delete, health) and new (GET /frameworks, POST /frameworks/{id}/smoke-test, POST /estimate, POST /experiments/{id}/cancel, GET /experiments/{id}/export, GET and PUT /settings)
- Authentication/authorization (MVP model; optional shared token via environment variable)
- Request/response formats (experiment configuration with classic models, AutoML frameworks, budget, cores, memory, seed and priorities)
- Validation and error conventions (one consistent error shape; clear messages for dataset validation rules)

- Long-running job semantics (accepted then polled; statuses including interrupted and cancelled)
- Pagination/filtering (experiments history, results tables)
- External APIs and integrations (OpenML, UCI, Kaggle for datasets; AutoML frameworks through the worker, not through HTTP)
- Webhooks and verification (state that none are needed)
- Rate limiting (concurrency limit of one running experiment, upload size limit, request limits)
- Required environment variables (database URL, data and environments directories, upload limit, optional API token, Kaggle credentials)
- Data privacy and failure handling (no dataset rows in logs, behavior when a framework environment is missing, API restart during a run)

RULES:
Define only required APIs. Never expose secrets client-side. Enforce authorization server-side. Keep conventions consistent and integrations fault-tolerant. Download endpoints may serve only registered artifacts, never arbitrary paths.

OUTPUT:
Return only clean Markdown ready to save as API.md. Do not write code.

6. AGENTS.md / Coding Rules

You are a senior software engineer and AI coding-agent specialist experienced in maintaining large production codebases.

CONTEXT:
Use PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md and API.md as the project source of truth. An AGENTS.md already exists in the repository. Read it first, keep every rule that is still correct, and produce a merged and improved version rather than overwriting it blindly.

TASK:
Create AGENTS.md containing the rules the coding agent must follow throughout development.

Define rules for:
- Code quality and naming (Python 3.11+, type hints, docstrings on public functions, ruff formatting)
- Project/folder structure (business logic in src/acbs, none in API routes or frontend scripts; frontend under web/ with pages, css/, js/ and vendor/)
- Component reuse (shared UI components, one Plotly template, one API client)
- Dependency management (nothing added without justification and an entry in pyproject.toml; AutoML frameworks only inside their own isolated environments, never in the main environment; never pip install the unrelated package named auto-sklearn2)
- Type safety (pydantic models for configuration, tasks, outcomes and API schemas)
- Error handling (a failed or timed-out framework never stops a run; record status and reason)
- Security and secrets (no shell=True, framework identifiers only from a whitelist, no secrets in code or logs, no data rows in logs)
- Database/API access (through the repository layer and API client only; Alembic for every schema change)
- Benchmark integrity rules (no preprocessing outside pipelines, single seed, identical folds, central scoring from predictions, sequential framework runs, never fabricate or hardcode

benchmark results)
- Framework adapters (read the documentation of the pinned version before writing or changing an adapter; these APIs change between releases)
- Testing (pytest with the FakeAdapter by default; real-framework tests behind a marker; tests for every bug fix)
- Documentation (update docs and README with every behavior change)
- Git/change discipline (small focused commits, run pytest and ruff before committing, no unrelated changes)

RULES:
The agent must inspect existing code before changing it, follow the project documents, reuse existing patterns, avoid unnecessary rewrites/dependencies, never expose secrets, and avoid unrelated changes.

OUTPUT:
Return only clean Markdown ready to save as AGENTS.md.

7. Implementation Planning Prompt

You are a senior Technical Lead responsible for planning production software development.

CONTEXT:
Read PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md before planning. The v1 classic-tier system already exists with 28 passing tests, so plan only the delta to v2 plus housekeeping. Two people will work in parallel. Target duration: about 7 weeks from 26 October 2026 (my estimate; I will confirm the real deadline).

TASK:
Create a detailed implementation plan for this project WITHOUT writing application code. Break development into small, logical phases. Suggested phase skeleton, to be adjusted only if the documents justify it:

Phases to consider:
- A. Housekeeping: commit pending changes, fix deprecation warnings, initial Alembic migration
- B. Compatibility spike: find out which AutoML frameworks install and run on this machine before any adapter code is written
- C. Adapter layer: typed task/outcome contract, isolated runner with hard timeouts, FakeAdapter, then adapters in three groups, then a conformance suite
- D. Engine integration: two tiers on the same split and seed, cost estimator, CLI options
- E. Recommender upgrade: both tiers, AutoML-worth-it analysis, tie handling, explanations
- F. Storage and API changes for v2
- G. Frontend pages (HTML/CSS/JS) and charts (follow the 9-page design)
- H. Experiments and analysis for the research paper
- I. Model export (optional)
- J. Docker, documentation and review material

For each phase define:
- Goal
- Features/tasks
- Files or modules likely affected

- Dependencies/prerequisites
- Database/API changes
- Tests required
- Completion criteria
- Owner (Person A: ML and framework core; Person B: platform, API and UI) and rough duration

RULES:
Build foundations before dependent features. Keep tasks small enough to implement and verify independently. Follow the architecture and MVP scope. Do not introduce features or technologies not specified in the project documents. Schedule long-running experiments early so they run in the background while the UI is built.

OUTPUT:
Return an ordered implementation plan with clear phases and tasks. End with the recommended first task.

8. Project Setup Prompt

You are a senior full-stack engineer responsible for establishing production-ready project foundations.

CONTEXT:
Read PRD.md, DESIGN.md, ARCHITECTURE.md and AGENTS.md before making changes. The project already exists and runs, so this is an alignment task: bring the foundations in line with the approved v2 architecture without breaking the 28 existing tests.

TASK:
Inspect the current repository, list the gaps against the approved architecture, then fix them.

Set up:
- Required framework and dependencies (pyproject.toml extras and dev tools; no AutoML framework in the main environment)
- Folder structure (add automl/, a separate worker package, envs/ configuration, configs/ for framework information; web/ with css/, js/components and js/pages)
- Configuration (YAML defaults and a settings loader)
- Environment variable structure (.env.example with database URL, data and environments directories, upload limit, optional API token)
- Base routing (9 HTML pages with a shared navigation shell and placeholders; FastAPI serving web/ and the router layout under /api)
- Core styling/design system (tokens.css, base and component CSS, shared Plotly layout, shared JS components skeleton)
- Shared utilities/components (timezone-aware time helper replacing datetime.utcnow, logging setup)
- Development scripts (Makefile or equivalent: install, test, lint, api, ui, setup-envs, smoke-test-envs)
- Linting/formatting/type checking (ruff, mypy or the type checker in AGENTS.md, pre-commit)
- Alembic initial migration for the existing schema and migration on startup
- Fix known warnings: datetime.utcnow usage and scikit-learn n_jobs deprecation for LogisticRegression
- Git-ready project structure (.gitignore for data, results, .envs and databases)

RULES:
Follow the documented architecture exactly. Inspect the existing project first. Do not add unnecessary packages or build application features yet. Never hardcode secrets. Keep the project runnable after setup.

OUTPUT:
Implement the setup, verify it runs successfully (tests and linter green), and briefly report what was created, commands used and any decisions requiring approval.

9. Feature Development Prompt

You are a senior full-stack engineer working inside an existing production codebase.

CONTEXT:
Follow PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md. Implement only this approved task:

FEATURE F1: AutoML adapter contract and isolated runner. Define typed AutoMLTask and AutoMLOutcome models; create a small separate worker package that loads training and test data, runs one framework adapter, measures fit time and prediction time, and writes predictions and an outcome file; create a runner in the main app that launches the worker in the framework's own environment as a subprocess with a hard timeout (budget plus grace), kills the whole process group on timeout, records peak memory, and returns status ok, failed, timeout or unavailable; add central scoring of the returned predictions with the existing metrics; add a FakeAdapter (ok, slow, crash, wrong class order) for tests.

TASK:
First inspect the relevant existing code and dependencies. Briefly determine the implementation approach, then implement the feature completely.

Requirements:
- Follow existing architecture and design system
- Reuse existing components/utilities (metrics, timing, statuses)
- Handle loading, empty, success and error states
- Validate inputs and enforce authorization (framework identifiers only from a whitelist; never build shell commands from user input)
- Update database/API code when required
- Add or update relevant tests
- Keep changes focused

RULES:
Do not rewrite unrelated code or introduce unnecessary dependencies. Do not silently change requirements. If a blocking ambiguity exists, stop and ask.

OUTPUT:
Implement, test, and report changed files, tests performed, and any remaining issues.

FEATURE QUEUE (reuse this prompt, replacing the feature text above with one item at a time):
- F2: Adapters for FLAML, MLJAR, GAMA and TPOT in the worker package, with integration tests behind an automl marker

- F3: Adapters for AutoGluon, H2O AutoML (including guaranteed JVM shutdown), LightAutoML and NaiveAutoML
- F4: Adapters for Auto-sklearn and Auto-sklearn 2 (Linux only, unsupported status elsewhere) and the adapter conformance suite
- F5: Two-tier benchmark engine with sequential framework runs, the cost estimator and the CLI automl options
- F6: AutoML-aware recommender: interpretability scores, AutoML-worth-it analysis, tie handling, new explanation templates
- F7: One frontend page (HTML, CSS and JS module) per feature, in this order: Datasets, New Benchmark, Run Monitor, Results, Model Detail, History, Frameworks, Home, Methodology
- F8: Model export (joblib bundle for classic models, native artifact zip for frameworks)
- F9: Full experiment runner and analysis notebook producing the figures for the paper

10. Database & Backend Connection Prompt

You are a senior backend engineer and database specialist experienced in production systems.

CONTEXT:
Follow DATABASE.md, ARCHITECTURE.md, API.md and AGENTS.md as the source of truth. A working SQLAlchemy layer and FastAPI backend already exist; extend and harden them for v2.

TASK:
Connect and implement the application's backend and database for the v2 design.

Do the following:
- Configure the database securely (SQLite by default, PostgreSQL through the database URL environment variable)
- Create the approved schema and migrations (initial migration for existing tables, then the v2 migration; run migrations on startup)
- Implement models/data access through the repository layer
- Implement required backend services/API endpoints from API.md (frameworks, estimate, cancel, export, settings, rerank for both tiers)
- Run experiments in a separate worker process with progress persisted in the database; mark experiments interrupted after a restart; make cancel stop the running framework process
- Add validation and authorization (including the optional shared token)
- Handle transactions and errors (partial results are kept when a framework fails)
- Add appropriate indexes
- Add seed/development data if specified (built-in dataset registry)
- Add tests for critical database and backend behavior using the FakeAdapter

RULES:
Never hardcode credentials. Follow the approved schema. Do not modify database structure without justification. Enforce authorization server-side. Keep secrets in environment variables.

OUTPUT:
Implement and verify the connection and core backend functionality. Report migrations, endpoints/services created, tests run, and any issues requiring approval.

11. Authentication & Authorization Prompt

You are a senior security-focused full-stack engineer specializing in production authentication systems.

CONTEXT:
Follow PRD.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md. The approved MVP model is a single-user tool with no accounts. Access control therefore means protecting the API when it is reachable from a network, and protecting the server from unsafe input. If PRD.md or ARCHITECTURE.md specify accounts instead, follow them and stop to ask me before building anything beyond what they define.

TASK:
Implement secure access control for this project according to the approved model.

Cover:
- Default exposure: the API and UI bind to localhost unless the user explicitly configures otherwise
- Optional shared API token read from an environment variable; when set, every API route except health requires it
- Protected routes and APIs (all except health when a token is set)
- Token handling (compare in constant time, never log it, never send it to the browser from server configuration)
- Resource access checks: download and export endpoints serve only registered datasets, reports and model artifacts by identifier, never by user-supplied path
- Unauthorized/forbidden states (clear 401 and 403 responses and a matching UI message)
- CORS allow-list and production URLs
- Upload safety as access control: file size limit, allowed extensions, sanitized file names, content validated by parsing, no execution or deserialization of uploaded files
- Roles: document that there is one role in the MVP
- Relevant tests

RULES:
Use the access approach defined in ARCHITECTURE.md. Never store or log tokens insecurely. Never trust client-side checks. Enforce permissions server-side. Do not expose secrets. Do not add accounts, passwords or sessions unless the project documents require them.

OUTPUT:
Implement, test and report the access flow, protected resources, changed files and any security concerns, including a short note on what would be needed to support multiple users.

12. MCP Connection Prompt

You are a senior AI engineer experienced with MCP servers and AI coding agents.

CONTEXT:
The project requires this MCP capability:

(a) A library-documentation MCP server (for example Context7) so the agent reads the current documentation of the pinned versions of AutoGluon, Auto-sklearn, FLAML, GAMA, H2O, LightAutoML, MLJAR, NaiveAutoML, TPOT, Optuna, FastAPI and Plotly.js instead of guessing.

These APIs change between releases and wrong guesses cost hours.

(b) A browser-automation MCP server (for example Playwright) so the agent can run the web app and test the full user journey: upload, configure, run, monitor, results.

Follow AGENTS.md and the existing project architecture. The examples named are suggestions; verify what is actually available in my environment before configuring anything.

TASK:
Connect and configure the required MCP servers for the coding environment.

First determine:
- What tools/resources each MCP exposes
- Which capabilities this project actually needs (documentation lookup by library and version; page navigation, form filling and screenshots for the UI)
- Required configuration and permissions
- Required environment variables or credentials

Then:
Configure the connection and verify that the required tools work: fetch the documented fit signature of one AutoML framework through the documentation server, and open the running web app and read the page title through the browser server.

RULES:
Follow least-privilege access. Never expose credentials or commit secrets. Do not enable unnecessary tools. Do not modify application architecture unless required. Explain any permissions or configuration that require manual approval. If a capability is not available, say so and continue without it.

OUTPUT:
Report the MCP connection status, available relevant tools, configuration performed, verification results and any manual steps required.

13. External API / Service Integration Prompt

You are a senior integration engineer experienced in building reliable production integrations.

CONTEXT:
Follow API.md, ARCHITECTURE.md, DATABASE.md and AGENTS.md. This project has two kinds of external integrations: dataset sources and AutoML frameworks. Frameworks are integrated as isolated subprocess adapters, not as network services.

TASK:
Integrate this external service (one target per run of this prompt):

TARGET 1: dataset sources (OpenML, UCI repository and Kaggle) through the dataset registry and loader, with local caching as Parquet.

TARGET 2 to 4: the AutoML frameworks, in the three groups defined in the feature queue (FLAML, MLJAR, GAMA, TPOT; then AutoGluon, H2O, LightAutoML, NaiveAutoML; then Auto-sklearn and Auto-sklearn 2). Use the documentation of the pinned version of each framework before writing its adapter.

Implement:
- Secure authentication/configuration (Kaggle credentials from environment variables; none for the frameworks)
- Required API operations (download and load datasets; for each framework: fit with a time budget, predict probabilities, describe the final model, report the framework version)
- Request/response handling (adapter contract: train data, target, budget, cores, memory, seed, metric; outcome with ordered class labels and probabilities)
- Input/output validation (probability columns must match the class order and sum to about 1; mismatches mark the run failed)
- Error handling and timeouts (budget plus grace; kill the whole process group; H2O must always shut down its JVM)
- Rate-limit handling where relevant (dataset downloads)
- Retries only where safe (dataset downloads may retry; a failed framework run must never be retried automatically because it would distort timing)
- Webhooks and signature verification (not required; state that)
- Logging without exposing sensitive data (never log dataset rows)
- Tests for success and failure cases (success, missing environment, crash, timeout, wrong class order)

RULES:
Never expose API keys client-side or hardcode secrets. Follow the provider's documented API behavior. Make failures graceful and prevent duplicate operations where relevant. Do not add unrelated functionality. If a framework cannot be installed here, record the exact error and mark it unsupported instead of working around it.

OUTPUT:
Implement and verify the integration. Report configuration, environment variables required, files changed, tests performed and any remaining limitations.

14. Testing & QA Prompt

You are a senior QA engineer and test automation specialist responsible for production software quality.

CONTEXT:
Read PRD.md, API.md, ARCHITECTURE.md and the current implementation before testing. 28 tests exist for the classic tier; extend coverage to the v2 behavior.

TASK:
Create and execute a practical test strategy for the implemented functionality.

Test:
- Core user journeys (upload, configure, run, monitor, read recommendation, re-rank, export)
- Happy paths
- Edge cases (single-feature dataset, 10 minority samples, all-categorical data, constant column, string labels, numeric target with 50 or more unique values rejected as regression, missing values, duplicate column names)
- Invalid inputs (missing target, one class, fewer than 50 rows, unsupported file, oversized file)
- Benchmark integrity (no data leakage, same seed gives same result, identical folds across models, central scores match a manual scikit-learn calculation)

- Framework behavior (timeout is killed and reported, crash is isolated, missing environment gives unavailable, wrong class order is detected, Linux-only framework gives unsupported elsewhere)
- Adapter conformance suite for every installed framework on three datasets (binary numeric, multiclass with missing values and categoricals, 5 percent minority)
- Authentication and permissions (optional token, path traversal on downloads)
- API/database failures (restart during a run marks the experiment interrupted, migrations apply to a fresh database)
- Loading, empty and error states in the UI
- Important integrations (dataset loaders)
- Responsive behavior at 1024 px width and keyboard-only navigation
- Performance smoke test and a golden-results regression test on a tiny dataset

Add appropriate tests:
Add unit, integration and end-to-end tests for critical behavior. Use the FakeAdapter by default and put real-framework tests behind a marker so the normal test run stays fast.

RULES:
Prioritize business-critical paths over superficial coverage. Do not modify production behavior just to make tests pass. Investigate failures rather than masking them. Do not report benchmark numbers you did not run.

OUTPUT:
Report tests created/run, passed and failed cases, bugs discovered, severity, and recommended fixes.

15. Debugging / Root Cause Prompt

You are a senior debugging engineer responsible for diagnosing production-quality software issues.

CONTEXT:
Follow the project's existing architecture, code and AGENTS.md rules. Known risk areas in this project, to check early when relevant: probability columns in a different class order than the labels; a framework process or H2O JVM left running after a timeout and polluting the next run; wrong timing because runs overlapped; a framework installed into the wrong environment; version-specific API changes in AutoML frameworks; Auto-sklearn on a non-Linux system; preprocessing applied outside a pipeline; progress stuck after an API restart.

PROBLEM:
[ERROR / BUG / LOG / SCREENSHOT] (paste the real error, the experiment id and the framework name here)

TASK:
Investigate the issue systematically.

1. Reproduce or trace the failure.
2. Inspect relevant code, logs, data and dependencies.
3. Identify the actual root cause.
4. Explain why it happens.
5. Implement the smallest correct fix.

6. Test the fix and check for regressions.

RULES:
Do not guess or randomly patch code. Do not hide errors, weaken validation, remove tests, or change unrelated functionality. Preserve existing behavior unless the bug requires otherwise. If evidence is insufficient, state what is missing. Add a regression test for the bug.

OUTPUT:
Report root cause, fix implemented, files changed, tests performed and any remaining risk.

16. Security Audit & Hardening Prompt

You are a senior application security engineer specializing in production web applications.

CONTEXT:
Audit the current project using PRD.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md. This application accepts uploaded datasets (possibly containing health or financial data), runs third-party AutoML frameworks as subprocesses, generates HTML and PDF reports from user-supplied names, and can export serialized model artifacts.

TASK:
Perform a security audit and fix vulnerabilities you find.

Check:
- Authentication and authorization (optional token enforcement, localhost binding by default)
- Secrets and environment variables (Kaggle credentials, API token, nothing in logs or reports)
- API and database access (parameterized queries through SQLAlchemy only)
- Input validation and sanitization (file name, dataset name, column names, target name)
- Injection and XSS risks, especially: dataset, column and class names rendered in HTML reports (autoescaping), unsafe innerHTML usage, missing Content-Security-Policy and unvalidated API token handling in the frontend, and external resource loading during PDF generation
- Subprocess execution (no shell=True, framework identifiers from a whitelist, sanitized environment, resource limits, process-group kill)
- CSRF where applicable
- File uploads (size limit, extension and content checks, decompression bombs, path traversal, no deserialization of uploaded files)
- Serialized artifacts (joblib or pickle files must only be produced and loaded by this application; warn users that loading untrusted artifacts executes code)
- Local network ports opened by frameworks (for example H2O and Dask): verify they bind to localhost only
- Rate limiting and abuse (concurrency limit, large uploads, repeated experiment starts)
- Sensitive data exposure (dataset rows in logs, error messages, reports; deletion actually removes files)
- Dependencies (pinned versions, known vulnerabilities in the main environment)
- Error handling and logging
- Client/server trust boundaries

RULES:
Do not weaken functionality or security controls to hide issues. Do not expose secrets. Prioritize exploitable and high-impact issues first. Make fixes minimal and compatible with the existing architecture.

OUTPUT:
Report findings by severity, fixes implemented, tests performed, and any risks that remain.

17. Code Review & Refactoring Prompt

You are a senior software engineer reviewing a production codebase before release.

CONTEXT:
Review the current implementation against PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md.

TASK:
Perform a thorough code review for:

- Architecture violations (ML logic in the UI or API routes; framework imports in the main application package; adapters bypassing the runner)
- Bugs and incorrect logic (normalization, ranking, ties, class order, timing)
- Duplicate or dead code (two scoring paths, unused UI helpers, leftover v1 code)
- Poor abstractions
- Overly complex code
- Inconsistent patterns (statuses, error shapes, naming)
- Unnecessary dependencies
- Poor error handling (swallowed exceptions, leaked processes)
- Maintainability issues (hardcoded budgets, paths or framework lists instead of configuration)
- Technical debt
- Performance concerns (reloading large datasets repeatedly, excessive polling, repeated API calls and oversized Plotly payloads in the frontend)

Fix issues that are clearly justified and safe to change.

RULES:
Do not rewrite working code merely for style. Preserve behavior and follow existing architecture. Do not introduce unnecessary abstractions or dependencies. Test important changes.

OUTPUT:
Report critical findings, refactoring performed, files changed, tests run and remaining technical debt.

18. Deployment & Production Setup Prompt

You are a senior DevOps engineer experienced in deploying secure production applications.

CONTEXT:
Follow ARCHITECTURE.md, API.md, DATABASE.md and AGENTS.md. This application is local-first and runs long jobs (minutes to hours), so serverless hosting is unsuitable. Target: a single machine (a laptop, a lab server or one cloud virtual machine) running the API, the UI and a database, plus isolated environments for the AutoML frameworks. Auto-sklearn requires Linux; H2O requires Java.

TASK:
Prepare this application for a reproducible deployment.

Configure and verify:
- Production build (package build and pinned dependencies)
- Hosting/deployment environment (docker-compose with api, ui and database services; recommended resource limits for cores and memory)
- Framework runner backends: virtual environments or one Docker image per framework (Auto-sklearn on a Linux base, H2O with a JDK installed), selectable by configuration
- Environment variables and secrets (.env.example, nothing committed)
- Production database (PostgreSQL with a persistent volume)
- Database migrations (applied on startup, verified on a fresh database and on an existing one)
- Persistent volumes for datasets, results, reports and framework environments
- Domain and HTTPS where applicable (only if exposed beyond localhost; otherwise bind to localhost)
- CORS and production URLs
- Build/start commands
- Error handling (restart policy, interrupted experiments marked on startup)
- Required external services (Java for H2O, dataset source access)
- Health checks (API health and UI availability)
- Production configuration (log level, upload limit, concurrency limit, default budgets)

RULES:
Never commit secrets or use development credentials in production. Do not destroy existing data. Verify the production build before deployment. Follow the architecture's deployment strategy and flag anything requiring manual approval (hosting choice, public exposure).

OUTPUT:
Deploy or prepare the deployment, verify it by running one small benchmark end to end, and report configuration, environment variables required, migration status, deployment URL/status and any remaining manual steps.

19. Production Monitoring & Maintenance Prompt

You are a senior SRE responsible for keeping production applications reliable and observable.

CONTEXT:
Review the deployed application and its ARCHITECTURE.md before making changes. The scale is small (one to a few users, one experiment at a time), so monitoring must be simple, local and free of external services.

TASK:
Set up practical production monitoring for this application.

Cover where applicable:
- Application errors and exceptions (structured logs that include experiment and framework identifiers)
- Server/API logs (rotation, no dataset rows, no secrets)
- Database health (connection check, size)
- Uptime and availability (health and readiness endpoints)
- Performance (experiment duration versus the estimate, framework budget overruns)
- Failed external integrations (failure and timeout counts per framework, visible on the Frameworks page and in a summary table)
- Background jobs (heartbeat for running experiments, detection of stuck runs, cleanup of orphaned framework processes and H2O JVMs)
- Resource usage (peak memory per run, disk usage of datasets, results and framework environments, with a warning threshold)
- Critical user/business events (experiment started, finished, failed, cancelled)
- Alerts for actionable failures (shown as in-app banners and log entries; no paid alerting service)
- Backups and recovery verification (scheduled database backup, results directory copy, documented restore test)

RULES:
Do not collect unnecessary sensitive user data. Avoid noisy alerts. Never log secrets, passwords or sensitive tokens. Prefer simple monitoring appropriate for the project's actual scale.

OUTPUT:
Implement or configure the required monitoring and report what is tracked, alert conditions, backup/recovery status, and remaining observability gaps.

20. Final Production Audit Prompt

You are a senior engineering lead responsible for approving an application for production launch.

CONTEXT:
Audit the complete project against PRD.md, DESIGN.md, ARCHITECTURE.md, DATABASE.md, API.md and AGENTS.md. This is also a research deliverable, so reproducibility and honesty of results are launch criteria.

TASK:
Perform a final end-to-end production readiness audit.

Verify:
- All critical requirements and user journeys (a new user reaches a recommendation in about five clicks)
- UI/UX consistency (tier and status distinguishable without color; every page has loading, empty and error states)
- Functional correctness (re-ranking changes the recommendation without retraining; the AutoML-worth-it verdict matches its stated thresholds)
- Benchmark fairness and integrity (same split, seed, cores and memory for every contender; frameworks run one at a time; central scoring; no leakage)
- Framework coverage (every one of the 9 frameworks either passes the conformance suite or is documented as unavailable or unsupported with the exact reason)
- Authentication and authorization (the approved access model, download restrictions)

- Database integrity (migrations on a fresh and an existing database, restart recovery)
- API and integrations
- Security (upload handling, subprocess execution, serialized artifacts, report escaping)
- Tests and regressions (full suite green, linter clean)
- Performance (cost estimates are within a reasonable range of actual run time)
- Responsive behavior (1024 px) and accessibility (contrast, keyboard use)
- Error/loading/empty states
- Environment and deployment configuration (fresh clone to running app using the documentation only)
- Monitoring, backups and recovery (a restore test was performed)
- Reproducibility (rerunning an experiment with the same seed reproduces the figures; versions and hardware are recorded per run)
- Documentation (README, architecture, API, protocol and limitations; no claim without evidence)

RULES:
Do not assume something works because the code exists. Verify it where possible. Do not make risky changes automatically. Prioritize blockers and high-impact issues. Do not accept benchmark results that cannot be regenerated from the repository.

OUTPUT:
Give a launch verdict: READY, READY WITH WARNINGS, or NOT READY. List findings by severity, required fixes, tests performed, and anything that must be manually verified before launch.

Decisions That Need Your Approval

- Deadline: the 7-week plan and the milestones in prompt 7 depend on your real submission date.
- Single-user with no accounts (prompts 1, 3, 11). Add accounts only if the tool will be hosted for several people.
- Frontend is plain HTML, CSS and JavaScript with no build step (prompts 2, 3, 9). Confirm this over React or Vue; it keeps tooling minimal but means writing small shared components by hand.
- Move the API under /api and serve the static frontend from FastAPI at the same origin (prompts 3, 8). This changes existing v1 routes and needs your approval.
- Charts use Plotly.js with figure JSON generated by the backend, and all JS libraries are vendored locally (prompts 2, 3, 12). Optional: ESLint and Prettier for JS (prompt 8).
- Operating system: Auto-sklearn needs Linux, so decide between WSL2, Docker or documenting it as unsupported (prompts 3, 8, 18).
- Where it will run for the final demo: your laptop, a lab server or one cloud virtual machine (prompt 18).
- Default time budgets (60, 300 and 900 seconds) and the AutoML-worth-it thresholds, which are my proposals and should be confirmed (prompts 1, 7, 20).